

Financial Data Mining and Analysis

Chapter 9: Portfolio Theory

Agenda

- Introduction to portfolio theory
- A 2-stock portfolio
- N-stock portfolio
- Correlation versus diversification effect
- Producing a return matrix
- Generating an optimal portfolio based on Sharpe ratio, Treynor ratio, and Sortinor ratio
- Constructing an efficient frontier
- Modigliani and Modigliani performance measure (M2 measure)

Two key words

The key word for portfolio theory is
diversification

The key word for diversification is
correlation

Total, market, and firm-specific risks

Total risk have two components:

total risk = market risk + firm-specific risk

1) we use standard deviation of returns
to measure the total risk

2) we use beta to measure the market risk

Covariance's definition

Assume that we have n pairs of returns from stocks A and B:

$$\sigma_{A,B} = \sum_{i=1}^n (R_{A,i} - \bar{R}_A)(R_{B,i} - \bar{R}_B)$$

Three scenarios:

when the covariance > 0 , it suggest that wo stocks move in the same directions most of times.

when the covariance $= 0$, it suggest that wo stocks are not related to each other.

when the covariance < 0 , it suggest that wo stocks move in the opposite directions most of times

Correlation's definition

Assume that we have n pairs of returns from stocks A and B:

Step 1: we estimate two standard deviation and their covariance

Step 2: apply the following formula

$$\rho_{A,B} = \frac{\sigma_{A,B}}{\sigma_A \sigma_B}$$

$$-1 \leq \rho_{A,B} \leq 1$$

Correlation is better

- A correlations is a better measure than its corresponding covariance.
- For a covariance, we have three cases:
 - $\text{covar} > 0$, positively correlated
 - $\text{covar} = 0$, no correlation
 - $\text{covar} < 0$, no correlation
- When $\text{covar}(A,B) > \text{covar}(A,C)$, we can not claim that A is more closely correlated with B than with C.
- When $\text{corr}(A,B) > \text{corr}(A,C)$, we can claim that A is more closely correlated with B than with C.

Random time series with a correlation

This is a 2-step approach

Step 1: generate two random time series x_1 and x_2 with a zero-correlation

Step 2 apply the following formula

$$\begin{cases} y_1 = x_1 \\ y_2 = \rho x_1 + \sqrt{1 - \rho^2} * x_2 \end{cases}$$

Python code

```
import numpy as np
np.random.seed(123)
n=1000
rho=0.3
x1=np.random.normal(size=n)
x2=np.random.normal(size=n)
y1=x1
y2=rho*x1+np.sqrt(1-rho**2)*x2
print(np.corrcoef(y1,y2))
```

$$\begin{cases} y_1 = x_1 \\ y_2 = \rho x_1 + \sqrt{1 - \rho^2} * x_2 \end{cases}$$

2-stock variance formula

$$\sigma_{2-stock}^2 = w_A^2 \sigma_A^2 + w_B^2 \sigma_B^2 + 2w_A w_B \sigma_{A,B}$$

$$\sigma_{2-stock}^2 = w_A^2 \sigma_A^2 + w_B^2 \sigma_B^2 + 2w_A w_B \rho_{A,B} \sigma_A \sigma_B$$

$\sigma_{2-stock}^2$ variance of a 2-stock portfolio

w_A weight of stock A

w_B weight of stock B

$\sigma_{A,B}$ covariance between A and B

$\rho_{A,B}$ correlation between A and B

Example: 2-stock volatility

$$\sigma_{2-stock}^2 = w_A^2 \sigma_A^2 + w_B^2 \sigma_B^2 + 2w_A w_B \sigma_{A,B}$$

```
def var2stock(wA,varA,varB,covAB):  
    wB=1-wA  
    var=wA**2*varA+wB**2*varB**2+2*wA*wB*covAB  
    return(var)
```

```
In [8]: var2stock(0.6,0.2,0.23,0)  
Out[8]: 0.52
```

```
In [9]: var2stock(0.6,0.2,0.23,-0.22)  
Out[9]: 0.4144
```

```
In [10]: var2stock(0.4,0.4,0.23,-0.22)  
Out[10]: 0.4144
```

Var of 2-stock portfolio (2nd format)

```
def var2stock2(wA, stdA, stdB, corrAB):  
    wB=1-wA  
    cov=corrAB*stdA*stdB  
    var=wA**2*stdA**2+wB**2*stdB**2+2*wA*wB*cov  
    return(var)
```

```
In [12]: var2stock2(0.4,0.2,0.3,-1)  
Out[12]: 0.010000000000000002  
  
In [13]: var2stock2(0.4,0.2,0.3,0)  
Out[13]: 0.0388  
  
In [14]: var2stock2(0.4,0.2,0.3,0.8)  
Out[14]: 0.061840000000000006  
  
In [14]:
```

Formula to estimate variance and standard deviation

$$\left\{ \begin{array}{l} \mathbf{var} = \boldsymbol{\sigma}^2 = \frac{\sum_{i=1}^n (\mathbf{R}_i - \bar{\mathbf{R}})^2}{n-1} \\ \boldsymbol{\sigma} = \sqrt{\boldsymbol{\sigma}^2} \end{array} \right.$$

σ_{annual} , σ_{monthly} and σ_{daily}

- Conversion formulae

$$\begin{cases} \sigma_{\text{annual}}^2 = 252\sigma_{\text{daily}}^2 \\ \sigma_{\text{annual}}^2 = 12\sigma_{\text{monthly}}^2 \end{cases}$$

$$\begin{cases} \sigma_{\text{annual}} = \sqrt{252}\sigma_{\text{daily}} \\ \sigma_{\text{annual}} = \sqrt{12}\sigma_{\text{monthly}} \end{cases}$$

Minimization

```
from scipy.optimize import minimize

def myFunction(x):
    return (3.2+5*x**2)

#
x0=100

result= minimize(myFunction,x0,method='nelder-mead',tol=1e-6)

print(result)
```

```
In [14]: print(result)
         message: Optimization terminated successfully.
         success: True
          status: 0
           fun: 3.2
            x: [ 0.000e+00]
           nit: 31
          nfev: 62
    final_simplex: (array([[ 0.000e+00],
                           [-5.960e-07]]), array([ 3.200e+00,  3.200e+00]))

In [15]:
```

Lambda function

```
fun = lambda x: (x[0] - 10)**2 + (x[1] - 25)**2  
a=(10,25)  
fun(a)
```

In Python, the **lambda** keyword is a shortcut used to create small, **anonymous functions**.

Practice

- **Create a Lambda function for**

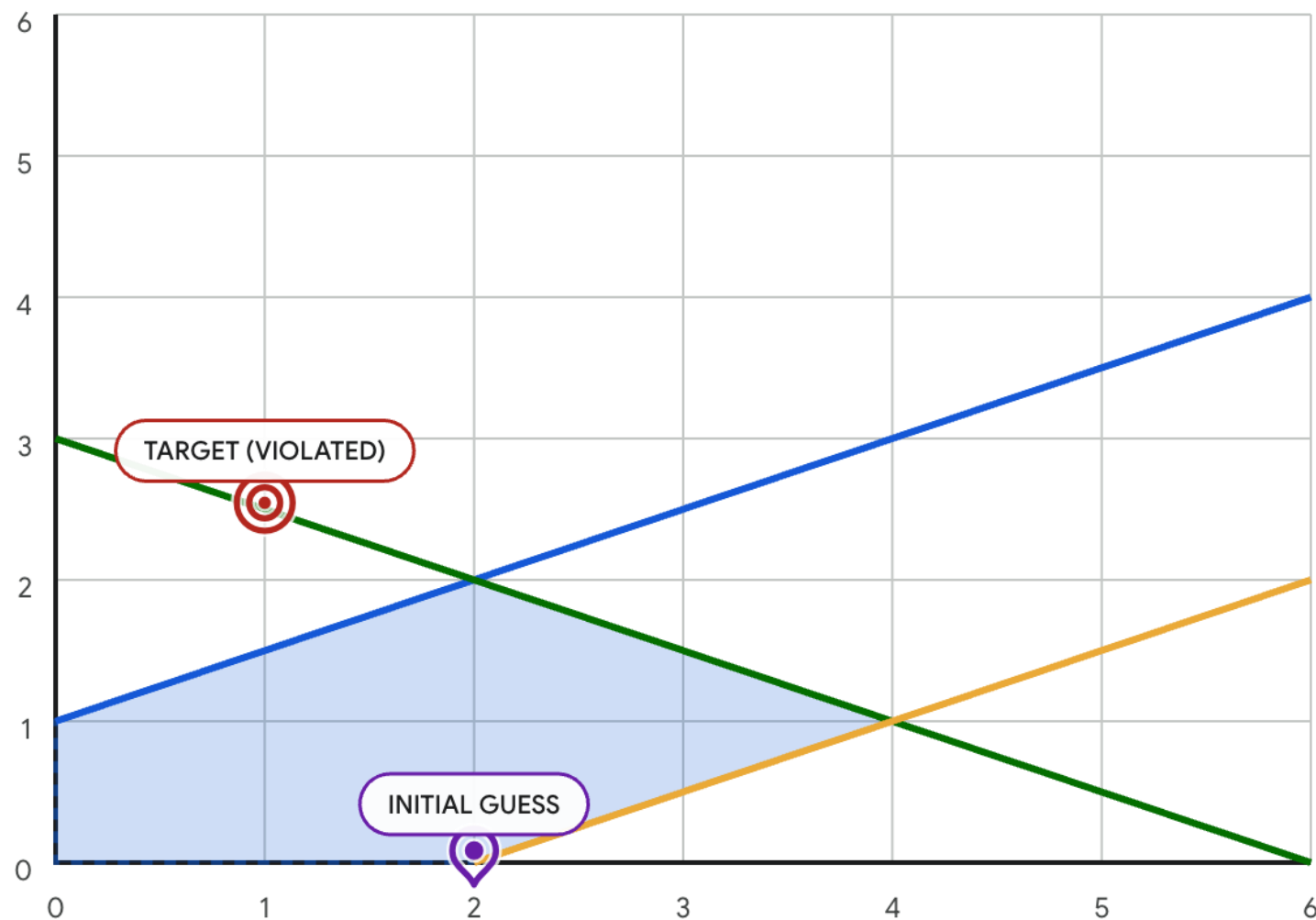
- $f(x_1, x_2, x_3) = (x_1 + 1) + (x_2 + 2)^2 + (x_3 + 3)^3$

Two input parameters and Lambda fun

```
from scipy.optimize import minimize
fun = lambda x: (x[0] - 1)**2 + (x[1] - 2.5)**2

bonds = ((0, None), (0, None))
cons = ({'type': 'ineq', 'fun': lambda x: x[0] - 2 * x[1] + 2},
        {'type': 'ineq', 'fun': lambda x: -x[0] - 2 * x[1] + 6},
        {'type': 'ineq', 'fun': lambda x: -x[0] + 2 * x[1] + 2})
#
result=minimize(fun, (2, 0), method='SLSQP',
bounds=bonds,constraints=cons)
```

Feasible Region Visualization



Annualized vol

- `volDaily=volStock(ticker,date1,date2)`
- `[*****100%%*****]` 1 of 1 completed

```
In [21]: import math
```

```
In [22]: volDaily*math.sqrt(252)
```

```
Out[22]: 0.20600327619620018
```

```
In [22]:
```

ret_monthly() function

```
def ret_monthly(ticker):  
    import numpy as np  
    import pandas as pd  
    import yfinance as yf  
    df=yf.download(ticker)  
    df["logret"]=np.log(df["Adj Close"].pct_change()+1)  
    df["yyyymm"]=df.index.year*100+df.index.month  
    retMonthly=np.exp(df.logret.groupby(df.yyyymm).sum())-1  
    retMonthly=pd.DataFrame(retMonthly)  
    retMonthly.columns=["ret_"+ticker]  
    return(retMonthly)
```

```
df=ret_monthly("ibm")  
df.head()
```

```
In [6]: df.head()  
Out[6]:
```

	ret_ibm
yyyymm	
196201	-0.052447
196202	-0.009091
196203	-0.007914
196204	-0.147817
196205	-0.135326

```
In [7]:
```

Covariance and correlation (coefficient)

$$\left\{ \begin{array}{l} \text{covariance } (\sigma_{A,B}) = \frac{\sum_{i=1}^n (R_{A,i} - \bar{R}_A)(R_{B,i} - \bar{R}_B)}{n-1} \\ \text{correlation } (R_A, R_B) = \rho(R_A, R_B) = \frac{\sigma_{A,B}}{\sigma_A * \sigma_B} \end{array} \right.$$

Python code

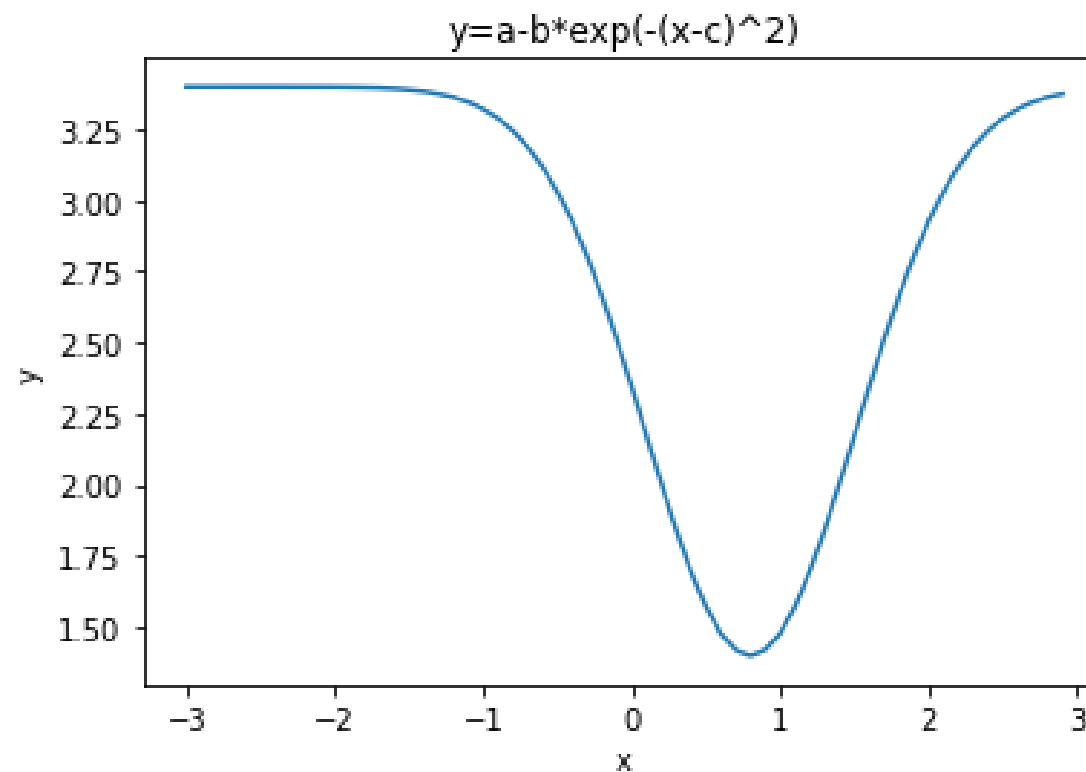
```
import numpy as np
from scipy import optimize
import matplotlib.pyplot as plt
# define a function
a=3.4
b=2.0
c=0.8
def f(x):
    return a-b*np.exp(-(x - c)**2)
```

```
x=np.arange(-3,3,0.1)
y=f(x)
plt.title("y=a-b*exp(-(x-c)^2)")
plt.xlabel("x")
plt.ylabel("y")
plt.plot(x,y)
plt.show()
# find the minimum
solution= optimize.brent(f)
print(solution)
```

Output: solution and graph

```
In [3]: print(solution)  
0.7999999995280379
```

```
In [4]:
```



Correlation matrix

```
> cor(data.frame(base[,2:n]))  
      X.GSPC  IBM    C  MSFT  DELL  
X.GSPC 1.000000 0.645613 0.676316 0.689405 0.662133  
IBM    0.645613 1.000000 0.325842 0.351916 0.543048  
C      0.676316 0.325842 1.000000 0.409860 0.409733  
MSFT   0.689405 0.351916 0.409860 1.000000 0.611168  
DELL   0.662133 0.543048 0.409733 0.611168 1.000000
```

Variance of a n-stock portfolio

Assume that our portfolio has n stocks.

$$R_p = \sum_{i=1}^n w_i \bar{R}_i = w_1 \bar{R}_1 + w_2 \bar{R}_2 + \cdots + w_n \bar{R}_n$$

Where w_1 is the weight of stock 1,
 $\bar{R}_1$ is the mean return of stock 1.

$$\sigma_p^2 = \sum_{i=1}^n \sum_{j=1}^n w_i w_j \sigma_{i,j}$$

where $\sigma_{i,j}$ is the covariance between stocks i and j

when $i=j$, then $\sigma_{i,j} = \sigma_i^2$

Assume that we have two stocks with variances of 0.0036 and 0.0576. The two stocks are negatively perfectly correlated. What is the weights (X_1, X_2) when the portfolio volatility is minimum? There exist at least three ways to get a solution.

Method 1: plug in given values into Equation (4) and set it equal zero where $x=x_1$ and $x_2=1-x$.

$$x_1^2 \sigma_1^2 + x_2^2 \sigma_2^2 + 2x_1 x_2 \sigma_{1,2} = x_1^2 \sigma_1^2 + x_2^2 \sigma_2^2 + 2x_1 x_2 \rho \sigma_1 \sigma_2 = 0$$

$$x^2 * 0.0036 + (1-x)^2 * 0.0576 - 2x(1-x)\sqrt{0.0036}\sqrt{0.0576} = 0$$

For a general formula of $ax^2 + bx + c = 0$, we have the following two solutions.

$$x = \frac{b \pm \sqrt{b^2 - 4ac}}{2a} \quad (6)$$

Based on a, b, and c, we have a solution of $x=80\%$. In other words, when $x_1=0.80$ and $x_2=0.2$, our 2-stock portfolio will be risk-free.

Performance measures (2)

- Concept #2: For a high-risk investment, investors demand a high return
- A general form

$$\textit{performance} = \frac{\textit{benefits}}{\textit{costs}} = \frac{\textit{return}}{\textit{risk}}$$

Performance measures (3): Sharpe Ratio

$$\textit{Sharpe Ratio} = \frac{\bar{R} - R_f}{\sigma}$$

Notations:

$\bar{R}$ is the mean return of the underlying stock of a portfolio

R_f is the risk-free rate or man risk-free rate

σ is the standard deviation of returns. This value represents the total risk of the underlying security or the portfolio.

Performance measures (4): Treynor Ratio

$$\textit{Treynor Ratio} = \frac{\bar{R} - R_f}{\beta}$$

Notations:

$\bar{R}$ is the mean return of the underlying stock of a portfolio

R_f is the risk-free rate or man risk-free rate

β is the market risk of the underlying security or the portfolio.

Performance measures (5): LPSD

- LPSD stands for Lower Partial Standard.
- It is a measure of downside risk that considers only the negative deviations from the mean or a benchmark. It is calculated as the square root of the average of the squared deviations of the observations that are below the mean or the benchmark.

LPSD formula

$$LPSD = \sqrt{\frac{\sum_{i=1}^n (R_i - R_{benchmark})^2}{m - 1}} \quad \text{where } R_i < R_{benchmark}$$

n is the number of observations (returns)

$R_{benchmark}$ is a benchmark return

m is the number of observations (returns) satisfying the condition, i.e., returns are less than the benchmark returns

If R_f is used as a benchmark:

$$LPSD = \sqrt{\frac{\sum_{i=1}^n (R_i - R_f)^2}{m - 1}} \quad \text{where } R_i < R_f$$

Utility function

- we could not quantify certain effects, such as happiness, willingness, risk preference, wellness, emotion, etc.
 - For example, your boss asks you to work extra hours on Saturday and promises you a bonus.
 - What is your trade-off between your value of leisure time and bonus?

$$U = u(bonus) - u(leisure)$$

Utility function with risk and return

- Assume that our utility function depends on two components: expected return and risk. It has the following form:

$$U = E(R) - \frac{1}{2}A\sigma^2$$

Financial Data Mining and Analysis

Chapter 10: Options and Futures

What is an option?

- For an option buyer, he/she has a right to buy or sell something
- For an option seller, he/she has an obligation to sell or buy something

What is a call option?

- A call option buyer has a right to buy an asset (commodity) in the future with a fixed price.
- A call option seller must sell an asset (commodity) in the future with a fixed price if the option buyer decides to exercise her option.

Payoff function for a call

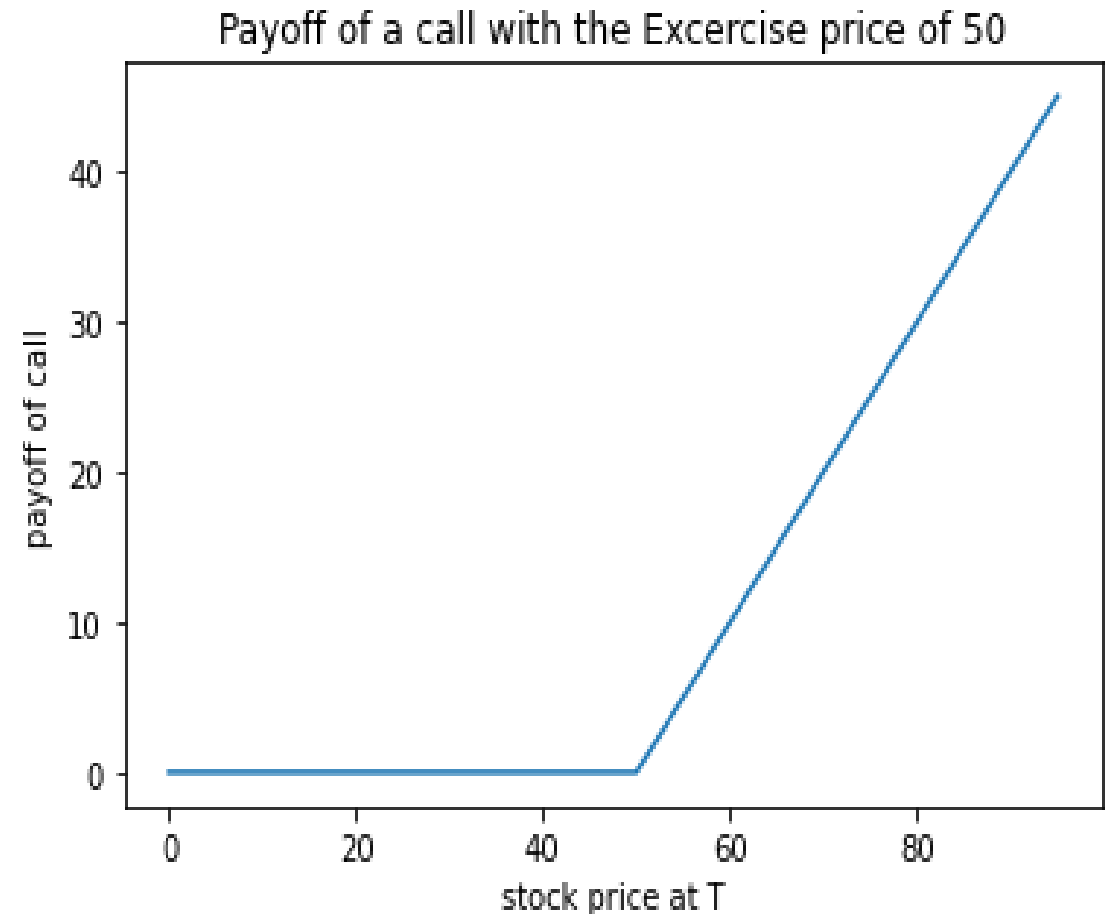
```
def payoff_call(sT,x):  
    return (sT-x+abs(sT-x))/2
```

where s_T is the stock price at time T

x is the exercise price of the call option

Graph of the payoff of a call option

```
import numpy as np
import matplotlib.pyplot as plt
#
def payoff_call(sT,x):
    return (sT-x+abs(sT-x))/2
sT=np.arange(0,100,5)
x=50
payoff=payoff_call(sT,x)
plt.plot(sT,payoff)
plt.title(f"Payoff of a call with the
Excercise price of {x}")
plt.xlabel("stock price at T")
plt.ylabel("payoff of call")
plt.show()
```



Profit/loss function of a call option **buyer**

- For a call option buyer, his/her profit or loss function is simply the difference between the payoff function and the cost of the call
- $\text{profit/loss (buyer)} = \text{payoff (buyer)} - \text{cost}$

Profit loss function for a call option

```
def profitLoss_call(sT,x,c):  
    profit=(sT-x+abs(sT-x))/2-c  
    return(profit)
```

sT=40

x=50

c=1.5

profitLoss_call(sT,x,c)

```
In [27]: profitLoss_call(sT,x,c)  
Out[27]: -1.5  
  
In [28]:
```

Buyer vs. seller

- The options transaction is a zero-sum game.
- Thus, the payoff and profit loss functions of a seller are the opposite of that of a buyer

Profit/loss function of a call option **seller**

- For a call option **seller**, his/her profit or loss function is the summation of the payoff function and the cost of the call
- $\text{profit/loss (seller)} = \text{payoff (seller)} + \text{cost}$
- $\text{profit/loss (seller)} = - (\text{profit/loss of the buyer})$

What is a put option?

- A put option buyer has a right to sell an asset (commodity) in the future with a fixed price.
- A put option seller must buy an asset (commodity) in the future with a fixed price if the option buyer decides to exercise her option.

Payoff function for a put option

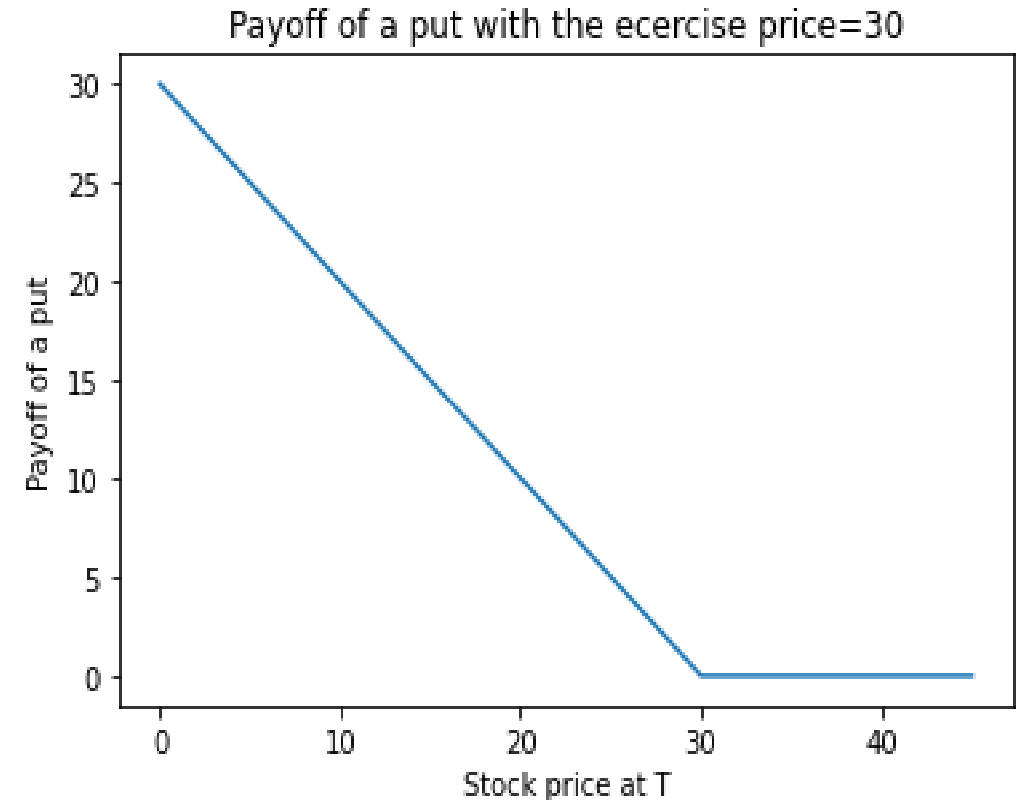
```
def payoff_put(sT, x):  
    payoff=(abs(x-sT)+x-sT)/2  
    return(payoff)
```

Graph of the payoffs for a put

```
import numpy as np
import matplotlib.pyplot as plt

def payoff_put(sT,x):
    payoff=(abs(x-sT)+x-sT)/2
    return(payoff)

sT=np.arange(0,50,5)
x=30
y=payoff_put(sT,x)
plt.plot(sT,y)
plt.title(f"Payoff of a put with the exercise price={x}")
plt.xlabel("Stock price at T")
plt.ylabel("Payoff of a put")
plt.show()
```



European vs. American options

- A European option can be exercised only on the maturity date
- An American option can be exercised any time before or on the maturity date

Cost of European and American options

- Other things equal, the cost of an American option is more valuable than a European option.

$$C_A \geq C_E$$

$$P_A \geq P_E$$

- C_A is the cost of an American call option
- C_E is the cost of an European call option
- P_A is the cost of an American put option
- P_E is the cost of an European put option

Positions, cash flow, rights and obligation

	Buyer (long position)	Seller (short position)	European Options	American Options
Call	A right to buy a security (commodity) at a pre-fixed price	An obligation to sell a security (commodity) at a pre-fixed price	Can be exercised on maturity day only	Can be exercised any time before or on maturity day
Put	A right to sell a security with a pre-fixed price	An obligation to buy		
Cash Flow	Upfront cash outflow	Upfront cash inflow		

Table 10.1 Long, short positions, initial cash flows, and right versus obligation

Black-Scholes-Merton call

$$d_1 = \frac{\ln\left(\frac{S_0}{X}\right) + \left(r + \frac{1}{2}\sigma^2\right)T}{\sigma\sqrt{T}}$$

$$d_2 = \frac{\ln\left(\frac{S_0}{X}\right) + \left(r - \frac{1}{2}\sigma^2\right)T}{\sigma\sqrt{T}} = d_1 - \sigma\sqrt{T}$$

$$C = S_0 N(d_1) - X e^{-rT} N(d_2)$$

Where S_0 is the current stock price

X is the exercise price

r is a continuously compounded rate

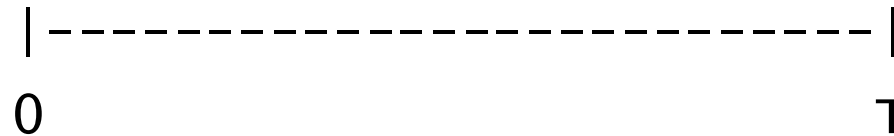
T is the time to maturity (in years)

σ is the volatility of the underlying stock

How to remember B-S call options

$$C = S_0 N(d_1) - X e^{-rT} N(d_2)$$

Step 1: draw a timeline



Step 2: at T, the payoff is

$$\text{Max}(s_T - X, 0)$$

Step 3: Drop 'max' and ()

$$s_T - X$$

Step 4: discount both s_T and X

s_0 $\leftarrow$ s_T
 $\exp(-rT)X$ $\leftarrow$ X
i.e., $s_0 - \exp(-rT)X$

Step 5: add distribution in front of those two values

$$\text{call} = S_0 * N() - \exp(-r*T)*X* N()$$

where $N()$ is the cumulative standard normal distribution.

Python code for Black-Scholes-Merton call

```
from scipy import stats
from math import log, exp, sqrt
#
def bs_call(S,X,T,r,sigma):
    d1=(log(S/X)+(r+sigma*sigma/2.)*T)/(sigma*sqrt(T))
    d2 = d1-sigma*sqrt(T)
    c=S*stats.norm.cdf(d1)-X*exp(-r*T)*stats.norm.cdf(d2)
    return(c)
```

```
bs_call(30,30,0.5,0.01,0.2)
```

```
In [2]: bs_call(30,30,0.5,0.01,0.2)
```

```
Out[2]: 1.762807270148281
```

```
In [3]:
```

Black-Scholes-Merton put

$$d_1 = \frac{\ln\left(\frac{S_0}{X}\right) + \left(r + \frac{1}{2}\sigma^2\right)T}{\sigma\sqrt{T}}$$

$$d_2 = \frac{\ln\left(\frac{S_0}{X}\right) + \left(r - \frac{1}{2}\sigma^2\right)T}{\sigma\sqrt{T}} = d_1 - \sigma\sqrt{T}$$

$$P = Xe^{-rT}N(-d_2) - S_0N(-d_1)$$

Where S_0 is the current stock price

X is the exercise price

r is a continuously compounded rate

T is the time to maturity (in years)

σ is the volatility of the underlying stock

Python code for Black-Scholes-Merton put

```
from scipy import stats
from math import log, exp, sqrt
#
def bs_put(S,X,T,r,sigma):
    d1=(log(S/X)+(r+sigma*sigma/2.)*T)/(sigma*sqrt(T))
    d2 = d1-sigma*sqrt(T)
    p=X*exp(-r*T)*stats.norm.cdf(-d2)-S*stats.norm.cdf(-d1)
    return(p)
#
bs_put(30,30,0.5,0.01,0.2)
```

```
In [7]: bs_put(30,30,0.5,0.01,0.2)
```

```
Out[7]: 1.6131816459287514
```

```
In [8]:
```

Put-Call Parity

Portfolio I: cash plus a call option Portfolio II: stock plus a put option

- Let's assume that the exercise price is x
- For Portfolio #1:
 - a) if $s_T > x$, we exercise the call
 - b) if $s_T < x$, we do nothing

The value of our portfolio is $\max(x, s_T)$

- For Portfolio #2:
 - a) if $s_T > x$, we let the put option expires
 - b) if $s_T < x$, we exercise the put

The value of our portfolio is $\max(x, s_T)$

- $C + PV(x) = S + P$

$$C + Xe^{-rfT} = P + S_0$$

Standard normal distribution

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

```
import scipy as sp  
sp.stats.norm.pdf(0)
```

```
from math import sqrt, pi  
1/sqrt(2*pi)
```

```
In [17]: from math import sqrt, pi
```

```
In [18]: 1/sqrt(2*pi)
```

```
Out[18]: 0.3989422804014327
```

```
In [19]:
```

Four functions for normal distributions

```
import numpy as np
import scipy.stats as stats
# 1) density
x=0
d=stats.norm.pdf(x)
d=round(d,5)
print(f"Density function when x is zero {d}")
# 2) cumulative density
c=stats.norm.cdf(x)
c=round(c,5)
print(f"Cumulative Density function when x is zero {c}")
```

```
# 3) inverse of the cumulative density
x2=stats.norm.ppf(c)
x2=round(x2,5)
print(f"Inverse of the Cumulative is {x2}")
# 4) Random values from a normal distribution
x=np.random.normal(3, 2.5, size=(2, 4))
print(x)
```

Output: 4 functions

```
Density function when x is zero 0.39894  
Cumulative Density function when x is zero 0.5  
Inverse of the Cumulative is 0.0  
[[5.02679695 2.17656889 1.87527934 1.64556034]  
 [4.34558887 3.49878009 0.22267819 3.51695692]]  
  
In [49]:
```

Formula for FV and PV formulae

$$fv = pv * (1 + R)^n$$

$$fv = pv * e^{R_c}$$

$$pv = \frac{fv}{(1 + R)^n}$$

$$pv = \frac{fv}{e^{R_c}}$$

From Rm to Rc

- The formula is given:

```
def Rm2Rc(Rm, freq):  
    from math import log  
    Rc=freq*log(1+Rm)  
    return(Rc)
```

convert a **discretely compounded interest rate** into an equivalent **continuously compounded interest rate**.

$$R_c = m * \log(1 + R_m)$$

We have the following results for a given annual percentage rate of 10% compounded annually.

```
In [23]: Rm2Rc(0.05,2)  
Out[23]: 0.09758032833886408
```

Verify: what is the future value of \$100 in 2 years given those two rates?

```
In [30]: 100*(1+0.05)**4  
Out[30]: 121.55062500000003  
  
In [31]: Rc=Rm2Rc(0.05,2)  
  
In [32]: from math import exp  
  
In [33]: 100*exp(Rc*2)  
Out[33]: 121.55062500000003
```

from APR (Annual Percentage Rate) to Rc

$$R_c = m * \log\left(1 + \frac{APR}{m}\right)$$

|

Implied volatility

- We know that when we have the following set of input values, the price of a call is \$2.77

```
In [3]: bs_call(40,42,0.5,0.1,0.2)
Out[3]: 2.277780329455348

In [4]:
```

- The last input is volatility which is estimated from a stock's returns
- Assume that we observe a call with \$2 with the same other input value. Then we ask what is the last input (volatility)?

Implied volatility (based on call)

```
from scipy import stats
from math import log,exp,sqrt
def implied_vol_call(S,X,T,r,c):
    for i in range(200):
        sigma=0.005*(i+1)
        d1=(log(S/X)+(r+sigma*sigma/2.)*T)/(sigma*sqrt(T))
        d2 = d1-sigma*sqrt(T)
        diff=c-(S*stats.norm.cdf(d1)-X*exp(-r*T)*stats.norm.cdf(d2))
        if abs(diff)<=0.01:
            print("          i,    std,    diff")
            sigma=round(sigma,7)
            diff=round(diff,7)
            return(i,sigma, diff)
```

```
In [29]: implied_vol_call(10,10,0.5,0.01,1.0)
          i,    std,    diff
```

```
Out[29]: (68, 0.345, 0.0064619)
```

```
In [30]:
```

Implied volatility using a put

```
import numpy as np
from scipy import stats
from math import log,exp,sqrt
```

```
def implied_vol_put_min(S,X,T,r,p):
    implied_vol=1.0
    min_value=100.0
    for i in np.arange(1,10000):
        sigma=0.0001*(i+1)
        d1=(log(S/X)+(r+sigma*sigma/2.)*T)/(sigma*sqrt(T))
        d2 = d1-sigma*sqrt(T)
        put=X*exp(-r*T)*stats.norm.cdf(-d2)-S*stats.norm.cdf(-d1)
        abs_diff=abs(put-p)
        if abs_diff<min_value:
            min_value=abs_diff
            implied_vol=sigma
            k=i
    put_out=put
    print('          k, implied_vol, put, abs_diff')
    return(k,implied_vol, put_out,min_value)
```

Output: implied vol based on a put

```
In [44]: implied_vol_put_min(10,10,0.5,0.01,2)
          k, implied_vol, put, abs_diff
Out[44]: (7274, 0.7275, 2.7315014647647105, 7.191319187160872e-05)

In [45]:
```

Logic behind various trading strategies

Assume that we expect a stock will increase its value in the near future (such as in 3 months). What might be our strategies?

- 1) buy the underlying stock (long)
- 2) long a future contract (long)
- 3) buy a call option
- 4) short (sell) a put option

Logic behind buying a put and a call at the same time

If expecting the price of a stock going up, we could long a call.

If expecting the price of a stock going down, we could long a put.

Assume that one company is going through a re-structuring process. If succeeding, its price will go up. However, if failed its price will fall.

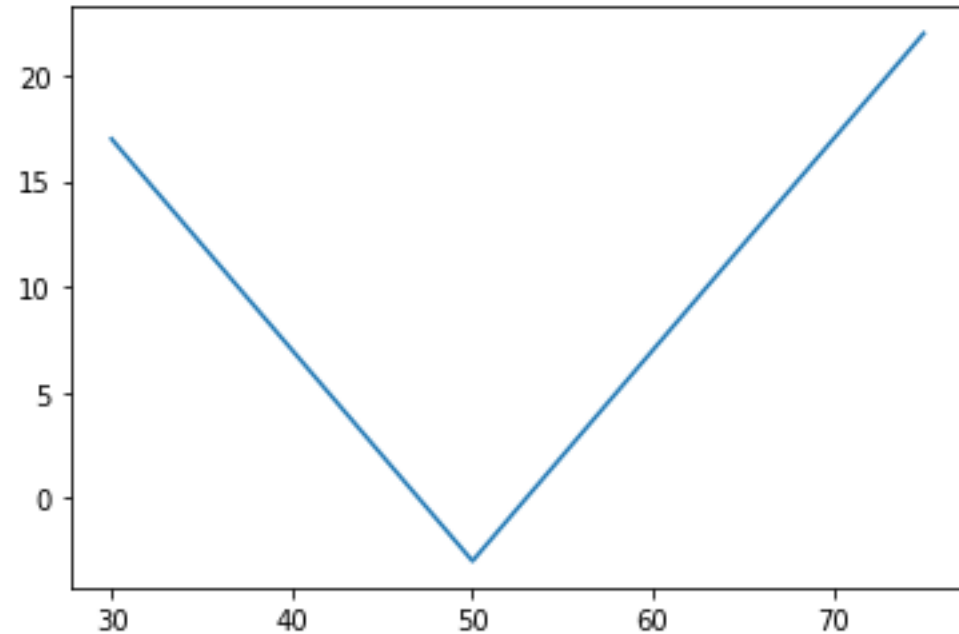
Thus, our trading strategy could be: buy a call and put at the same time.

Logic behind buying a put and a call at the same time

If the price rising, we will profit from our call.

If the price falling, we will profit from our put.

```
import numpy as np
import matplotlib.pyplot as plt
sT = np.arange(30,80,5)
x=50
c=2
p=1
straddle=(abs(sT-x)+sT-x)/2-c + (abs(x-sT)+x-sT)/2-p
plt.plot(sT,straddle)
```



Straddle: buy a call and a put

```
import numpy as np
import matplotlib.pyplot as plt
#
sT = np.arange(30,80,5)
x=50; c=2; p=1
straddle=(abs(sT-x)+sT-x)/2-c + (abs(x-sT)+x-sT)/2-p
y0=np.zeros(len(sT))

plt.ylim(-6,20)
plt.xlim(40,70)
plt.plot(sT,y0)
plt.plot(sT,straddle,'r')
plt.plot([x,x],[-6,4],'g-.')

plt.title("Profit-loss for a Straddle")
plt.xlabel('Stock price')
plt.ylabel('Profit (loss)')
plt.annotate('Point 1='+str(x-c-p), xy=(x-p-c,0), xytext=(x-p-c,10),
arrowprops=dict(facecolor='red',shrink=0.01),)
plt.annotate('Point 2='+str(x+c+p), xy=(x+p+c,0), xytext=(x+p+c,13),
arrowprops=dict(facecolor='blue',shrink=0.01),)
plt.annotate('exercise price', xy=(x+1,-5))
plt.annotate('Buy a call and buy a put with the same exercise price',xy=(45,16))
plt.show()
```

What is so-called Put-call parity?

- The formula is shown below.

$$C + Xe^{R_c T} = P + S$$

The present values of two portfolios are equal.

With the next two slides, the logic will be explained.

Put-call parity: portfolio #1

Portfolio 1: cash plus a call option

assume the exercise price is X

payoff at the maturity

if $s_T > X$ we exercise our call get stock

if $s_T \leq X$ we let the option expires

i.e., keep our money X

the value of our portfolio is

$\max(s_T, X)$

Put-call parity: portfolio #2

Portfolio 2: one share plus a put option

assume the exercise price is X

payoff at the maturity

if $s_T \leq X$ we exercise our put to have X

if $s_T > X$ we let the option expires

i.e., keep the stock

the value of our portfolio is

$$\max(s_T, X)$$

Put-call parity: conclusion

Since the final values of those two portfolio are the same, their present values should be the same as well.

$$\text{PV}(\text{portfolio \#1}) = \text{PV}(\text{portfolio \#2})$$

$$C + Xe^{R_c T} = P + S$$

Covered call

```
import numpy as np
import matplotlib.pyplot as plt
sT = np.arange(0,40,5)
k=15;s0=10;c=2
y0=np.zeros(len(sT))
y1=sT-s0                # stock only
y2=(abs(sT-k)+sT-k)/2-c # long a call
y3=y1-y2                # covered-call
plt.ylim(-10,30)
plt.plot(sT,y1)
plt.plot(sT,y2)
plt.plot(sT,y3,'red')
plt.plot(sT,y0,'b-.')
plt.plot([k,k],[-10,10],'black')
```

```
plt.title('Covered call (long one share and short one call)')
plt.xlabel('Stock price')
plt.ylabel('Profit (loss)')
plt.annotate('Stock only (long one share)',
xy=(24,15),xytext=(15,20),
arrowprops=dict(facecolor='blue',shrink=0.01),)
plt.annotate('Long one share, short a call',
xy=(10,4), xytext=(9,25),
arrowprops=dict(facecolor='red',shrink=0.01),)
plt.annotate('Exercise price= '+str(k),
xy=(k+0.2,-10+0.5))
plt.show()
```

CRR's procedure

Step 1. Draw a n-step tree

Step 2. At the end of n-step, estimate terminal prices.

Step 3. Calculate the option value at each node based on the terminal price, exercise, call or put.

Step 4. Discount it back one step, that is, from nth to nth-1, according to the risk neutral probability.

Step 5. Repeat the previous step until we find the final value at step 0.

The formulas for u, d, p are given here:

$$u = e^{\sigma\sqrt{\Delta t}}$$

$$d = \frac{1}{u} = e^{-\sigma\sqrt{\Delta t}}$$

$$a = e^{(r-q)\Delta t}$$

$$p = \frac{a-d}{u-d}$$

$$v_i = pv_{i+1}^u + (1-p)v_{i+1}^d$$

Options Greeks

The mathematical definitions of Greek letters for a European call and put are given in the following table:

Greek	A European call	A European put
Delta	$\Delta = N(d_1)$	$\Delta = N(d_1) - 1$
Gamma	$\Gamma = \frac{N'(d_1)}{S_0 \sigma \sqrt{T}}$	← same as the formula for the call
Theta	$\Theta = -\frac{S_0 N'(d_1) \sigma}{2\sqrt{T}} - rK e^{-rT} N(d_2)$	$\Theta = -\frac{S_0 N'(d_1) \sigma}{2\sqrt{T}} + rK e^{-rT} N(-d_2)$
Vega	$v = S_0 \sqrt{T} N'(d_1)$	← same as the formula for the call
Rho	$\rho = KT e^{-rT} N(d_2)$	$\rho = -KT e^{-rT} N(-d_2)$

Table 10.1 Mathematical definitions of Greek letters

Obviously, very few people can remember these formulae. Here is a very simple approach, based on their definition:

Greek	Description	Using call as an example
Delta	the change in an option value divided by the change in the underlying stock's price	$\Delta = \frac{C_2 - C_1}{S_2 - S_1}$
Gamma	the change in Delta divided by the change in the stock's price	$\Gamma = \frac{\Delta_2 - \Delta_1}{S_2 - S_1}$
Theta	the change in the option value divided by the change in the time	$\Theta = \frac{C_2 - C_1}{T_2 - T_1}$
Vega	the change in the option value divided by the change in the volatility	$v = \frac{C_2 - C_1}{\sigma_2 - \sigma_1}$
Rho	the change in the option value divided the by the change in the interest rate	$\rho = \frac{C_2 - C_1}{r_2 - r_1}$

Table 10.2 A simple approach to estimating Greek letters

One example: delta

```
from scipy import stats
from math import log,exp,sqrt
#
tiny=1e-9;S=40;X=40;T=0.5;r=0.01;sigma=0.2
def bsCall(S,X,T,r,sigma):
    d1=(log(S/X)+(r+sigma*sigma/2.)*T)/(sigma*sqrt(T))
    d2 = d1-sigma*sqrt(T)
    return S*stats.norm.cdf(d1)-X*exp(-r*T)*stats.norm.cdf(d2)

def delta1_f(S,X,T,r,sigma):
    d1=(log(S/X)+(r+sigma*sigma/2.)*T)/(sigma*sqrt(T))
    delta=stats.norm.cdf(d1)
    return(delta)

#
```

```
def delta2_f(S,X,T,r,sigma):
    s1=S
    s2=S+tiny
    c1=bsCall(s1,X,T,r,sigma)
    c2=bsCall(s2,X,T,r,sigma)
    delta=(c2-c1)/(s2-s1)
    return(delta)

#
delta1=round(delta1_f(S,X,T,r,sigma),6)
delta2=round(delta2_f(S,X,T,r,sigma),6)
print(f"delta (close form)={delta1}")
print(f"delta (tiny number)={delta2}")
```

```
In [183]: print(f"delta (close form)={delta1}")
delta (close form)=0.542235
```

```
In [184]: print(f"delta (tiny number)={delta2}")
delta (tiny number)=0.542238
```

```
In [185]:
```

Practice

- Try to code for Gamma

Delta	the change in an option value divided by the change in the underlying stock's price	$\Delta = \frac{C_2 - C_1}{S_2 - S_1}$
Gamma	the change in Delta divided by the change in the stock's price	$\Gamma = \frac{\Delta_2 - \Delta_1}{S_2 - S_1}$

1. Analytical Gamma (Closed Form)

The standard mathematical formula for the Gamma of both a European Call and Put is:

$$\Gamma = \frac{N'(d_1)}{S\sigma\sqrt{T}}$$

- 2. Numerical Gamma (Finite Difference)

To estimate the second derivative numerically, we use a technique called the **central difference method**. Instead of just looking forward (like what we did with Delta), we look slightly up and slightly down from the current stock price:

$$\Gamma \approx \frac{C(S + \Delta S) - 2C(S) + C(S - \Delta S)}{(\Delta S)^2}$$

Volatility smile

- A volatility smile is a common graph shape that results from plotting the strike price and implied volatility of a group of options with the same underlying asset and expiration date.
- The volatility smile is so named because it looks like a smiling mouth.

SQL basics with SQLite

- Have them imagine two separate spreadsheets that need to be combined:
 - **Table A (employees)**: Lists the employee's name and their department ID.
 - **Table B (departments)**: Lists the department ID and the department name.

SQL basics with SQLite

SQL JOIN Visualizer

ID	Name	Dept ID
1	Alice	10
2	Bob	20
3	Charlie	NULL

Dept ID	Dept Name
10	Engineering
20	Sales
30	Marketing

- Notice Charlie hasn't been assigned a department yet
- Notice Marketing has no employees yet

SQL basics with SQLite

Emp ID	Name	A.dept_id	B.dept_id	Dept Name
1	Alice	10	10	Engineering
2	Bob	20	20	Sales

Active Query

```
SELECT * FROM Employees INNER JOIN Departments ON Employees.dept_id = Departments.dept_id
```